Riftbound TCG privé

Rapport d'avancement — Étape 1 : Depot GitHub et CI DevSecOps

08/11/2026

Pour reprendre le projet dans une nouvelle conversation, donner ce PDF (ou le dépôt GitHub) à l'assistant et lui demander de lire **docs/ETAT_PROJET.md** puis de continuer à l'étape indiquée comme « prochaine étape ».

Prompt initial (verbatim)

Date : 2026-11-08 — Auteur : Sébastien Girard

> Je souhaite créer un tcg sur Rifbound mais de maniere privé géré par compte et pwd quand tu accèdes au site.
>
> Je souhaite utiliser cet api https://github.com/LouisCourrian/riftbound-cards ce qui me permet de rien stocker dans ma db.
>
> Ce sera hébérger sur cloudeflare gratuitement avec sqlite de cloudflare gratuit.
>
> Il faut pouvoir choisir le pack qu'on veut ouvrir (si c'est vendetta, unleashed ect). pouvoir échanger entre les joueurs et avoir 1 packs toutes les 10 mins. on peut en stocké 15. Avoir des raretés comme dans le vrai jeu de cartes avec les memes taux de drop les memes nbr de cartes par boosters ect.
>
> Faut que ce soit responsible sur tel pc tablette ect. Qu'on puisse utiliser la fct application sur navigateur comme ça c'est plus simple sur pc.
>
> On fait étape par étape. Dans un process de devsecops.
>
> On va utiliser GIthub ça va me permettre de switch sur mon pc portable. Je veux des rapports pdf d'avancement avec le prompt de base sur le fichier comme ça si je perds notre conv tu pourras pile reprendre ou on en est sans etre perdu.

Riftbound TCG privé — État du projet

Ce fichier est la source de vérité pour reprendre le projet. Le rapport PDF de chaque étape est généré depuis ce fichier et depuis `docs/PROMPT_INITIAL.md`.

Dernière mise à jour : 2026-11-08

Étape en cours : 1 — Dépôt GitHub, squelette, CI DevSecOps (terminée)

Prochaine étape : 2 — Compte Cloudflare, Worker et D1, premier déploiement automatique depuis la CI

Dépôt : https://github.com/Crousti-Bats/riftbound-tcg-prive (public, `main` protégée)

0. Reprendre sur un autre PC

```
git clone https://github.com/Crousti-Bats/riftbound-tcg-prive.git
cd riftbound-tcg-prive
npm ci
npm run dev        # site + API en local (http://localhost:5173)
npm test           # tests
```

Pré-requis : Git, Node.js 22 ou plus, GitHub CLI (`gh auth login`), Python avec `pip install reportlab` pour les rapports.

Méthode de travail : `git switch -c etape-N`, puis commit, puis `git push`, puis `gh pr create`. On merge quand la CI est verte.

1. Objectif

Un site privé de collection et d'échange de cartes Riftbound, accessible uniquement avec un compte et un mot de passe, hébergé gratuitement sur Cloudflare.

- Ouvrir des boosters en choisissant le set (Origins, Spiritforged, Unleashed, Vendetta, Radiance).
- Composition des boosters et taux de drop alignés sur le vrai jeu.
- Recevoir 1 booster toutes les 10 minutes, avec un stock de 15 au maximum.
- Échanger des cartes entre joueurs.
- Site responsive (téléphone, tablette, PC) et installable comme application (PWA).

2. Décisions d'architecture

Sujet	Décision	Raison
Hébergement	Un seul Cloudflare Worker qui sert le front (Static Assets) et l'API	Offre gratuite, un seul déploiement, aucun CORS
Base de données	Cloudflare D1 (SQLite), offre gratuite	Demandé. Limites : 5 Go, 5 M lectures/jour, 100 k écritures/jour
Données des cartes	`cards.json` de la dernière release de LouisCourrian/riftbound-cards, récupéré par le Worker et mis en cache (Cache API, 24 h)	Aucune carte stockée en base. D1 ne garde que des `cardCode` et des quantités
Images	Liens directs vers le CDN Riot (`imageUrl`)	Rien à héberger
Backend	TypeScript + Hono	Léger, prévu pour Workers
Frontend	React + Vite + TypeScript, `vite-plugin-pwa`	PWA installable, responsive
Authentification	Inscription uniquement sur invitation : l'admin génère des codes. Mots de passe hachés en PBKDF2-SHA256 (WebCrypto), sessions en base avec cookie `HttpOnly; Secure; SameSite=Strict`	Site privé, sans dépendance externe

Sujet	Décision	Raison
Aléatoire	`crypto.getRandomValues` côté serveur uniquement	Le client ne peut pas tricher
Timer des boosters	Calcul paresseux : `stock = min(15, stock_saved + floor((now - last_tick)/600 s))`, sans cron	Aucun coût, résultat exact
Échanges	Proposition, puis acceptation. Transfert atomique via `D1 batch()` après vérification de possession	Pas de duplication de cartes
Code source	GitHub (dépôt public, voir §5), synchronisé entre le PC fixe et le portable	Demandé

3. Données de cartes (analysées le 2026-11-08, release v2026-11-04)

1407 cartes. Raretés présentes dans les données : Common, Uncommon, Rare, Epic, Showcase.

Set	Code	Ouvrable en booster
Origins	ogn	oui
Proving Grounds	ogs	non (produit starter)
Spiritforged	sfd	oui
Unleashed	unl	oui
Vendetta	ven	oui
Radiance	rad	oui

On classe les variantes à partir de `cardNumber` :

- `NNN-TTT` avec NNN ≤ TTT : carte de base, avec sa rareté.
- `NNNa-TTT` : Alt Art (Showcase).
- `NNN-TTT` avec NNN > TTT : Overnumbered (Showcase).
- `NNN-star-TTT` : Overnumbered signée (Showcase).
- `spX-NNN` : Special Alt (Showcase).
- `rNN` et `rNNa` : Rune et Rune alt. `tNN` : Token ou Battlefield token.

4. Composition d'un booster (14 cartes, source officielle Riot)

Slot	Nb	Contenu
Common	7	Common
Uncommon	3	Uncommon
Foil	1	Common, Uncommon, Rare ou Epic en foil
Rare+	2	Rare, Epic, Alt Art, Special Alt, Overnumbered, Signée
Rune / Token	1	Rune, Rune alt ou Token

Taux par slot Rare+ (hypothèse initiale, configurable dans un fichier dédié) :

Résultat	Proba par slot	Source
Epic	13,4 %	Officiel : environ 1 booster sur 4 contient une Epic
Alt Art	4,2 %	Officiel : environ 2 par display de 24 boosters

Résultat	Proba par slot	Source
Overnumbered	0,69 %	Officiel : environ 1 pour 3 displays
Overnumbered signée	0,07 %	Officiel : environ 1 Overnumbered sur 10
Special Alt (sp)	0,05 %	Estimation (chase card)
Rare	le reste (environ 81,6 %)	

Slot Foil (estimation communautaire) : 75 % Common, 20 % Uncommon, 4 % Rare, 1 % Epic.

Slot Rune : 92 % Rune, 8 % Rune alt. Si le set n'a pas de rune, on tire un Token du set ou une Rune d'Origins.

5. Démarche DevSecOps

- Le dépôt est **public** (choix du 2026-11-08). Sur le plan gratuit, c'est la seule façon d'avoir la protection de branche, CodeQL et la push protection. Le site reste privé grâce à l'authentification.
- Branche `main` protégée par un ruleset « Protection main » : PR obligatoire (0 approbation, on est seul), pas de force-push ni de suppression, merge en squash uniquement, et 4 contrôles requis : `Lint, types, tests, build`, `Audit des dépendances`, `CodeQL (SAST)`, `Gitleaks (secrets)`.
- `.github/workflows/ci.yml` lance le lint (ESLint), le typecheck (tsc), les tests (Vitest), le build, `npm audit` et la dependency review sur les PR. Il vérifie aussi que `worker/worker-configuration.d.ts` est à jour.
- `.github/workflows/securite.yml` lance CodeQL (`security-extended`, JS/TS et Actions) et Gitleaks, à chaque PR, à chaque push et chaque lundi.
- Dependabot (npm et Actions, chaque semaine), alertes et correctifs automatiques activés. Secret scanning et push protection activés. Signalement privé de vulnérabilités activé (voir `SECURITY.md`).
- Chaîne d'approvisionnement : versions npm exactes (`.npmrc save-exact`), lockfile, `npm ci`, actions épinglées par SHA, `permissions` minimales.
- Override `minimatch@11.0.5` dans `package.json` : corrige GHSA-g3hv-75wg-m249 (ReDoS, moderate), remonté via eslint 10. On pourra le retirer quand eslint le corrigera.
- Les secrets (token Cloudflare) sont uniquement dans GitHub Secrets. Jamais dans le code.
- En-têtes de sécurité : CSP stricte, HSTS, X-Frame-Options DENY, Referrer-Policy.
- Rate limiting sur la connexion. Validation des entrées avec Zod. Requêtes SQL paramétrées uniquement.
- Le déploiement en production se fait automatiquement par la CI au merge sur `main`.

6. Feuille de route

#	Étape	Statut
0	Cadrage, architecture, analyse des données, système de rapports	Terminé
1	Outils (Git, Node), dépôt GitHub, squelette du monorepo, CI sécurité	Terminé
2	Compte Cloudflare, Worker et D1, premier déploiement "Hello" via la CI	À faire
3	Authentification : invitations, connexion, sessions, rate limit	À faire
4	Catalogue des cartes (fetch et cache du JSON, API sets et cartes)	À faire
5	Moteur de boosters (taux, tirage, tests statistiques) et timer 10 min / 15 max	À faire
6	Frontend : ouverture de booster animée, collection, filtres	À faire

#	Étape	Statut
7	Échanges entre joueurs	À faire
8	PWA (installable, hors-ligne partiel), responsive	À faire
9	Durcissement sécurité (revue, en-têtes, tests d'abus) et mise en prod	À faire

7. Actions manuelles en attente (utilisateur)

1. Créer un compte Cloudflare gratuit (https://dash.cloudflare.com/sign-up), s'il n'existe pas déjà.
2. Au début de l'étape 2, l'assistant lancera `npx wrangler login`, qui ouvre le navigateur pour autoriser l'accès.
3. Sur le portable : installer Git, Node.js LTS et GitHub CLI (`winget install --id Git.Git -e`, `OpenJS.NodeJS.LTS`, `GitHub.cli`), puis lancer `gh auth login`.

8. Journal

- **2026-11-08 — Étape 0.** Analyse de l'API (1407 cartes, 6 sets), recherche des taux officiels, choix de l'architecture, mise en place des rapports PDF (`tools/generate_report.py`).
- **2026-11-08 — Étape 1.** Dépôt public `Crousti-Bats/riftbound-tcg-prive` créé. Squelette : Worker Hono (`worker/`, route `/api/health`, en-têtes de sécurité, 3 tests), front React 19 et Vite 8 (`src/`), `@cloudflare/vite-plugin`, TypeScript strict, ESLint. CI et sécurité (CodeQL, Gitleaks, audit, dependency review, Dependabot), ruleset sur `main`. Le lint, les types, les tests et le build passent en local, avec 0 vulnérabilité.

Arborescence

```
worker/         API (Cloudflare Worker, Hono)       -> index.ts, security.ts, *.test.ts
src/            Front React (PWA à l'étape 8)       -> main.tsx, App.tsx, styles.css
public/         Fichiers statiques (_headers = en-têtes de sécurité du front)
docs/           PROMPT_INITIAL.md, ETAT_PROJET.md, rapports/*.pdf
tools/          generate_report.py
.github/        workflows ci.yml + securite.yml, dependabot.yml
wrangler.jsonc Configuration Cloudflare (D1 ajoutée à l'étape 2)
```